`scripts/pipeline/phase-06-docs.sh` — User Guide

Last verified: 2026-08-21 (feature `002-build-test-distribute-pipeline`, tasks T044 + T045) **Inheritance:** HelixConstitution §11.4.18 (script documentation mandate), §11.4.65 / `CM-UNIVERSAL-MARKDOWN-EXPORT-SYNC` (derived exports); Lava §6.J (Anti-Bluff)

Overview

The pipeline's **post-distribution** documentation-refresh phase (FR-013 / SC-006 "zero manual documentation follow-up required"). Two independent passes, in order:

PASS 1 (T045) — stale-documentation fixes

Corrects the two concrete staleness findings recorded in `specs/002-build-test-distribute-pipeline/research.md` **R-002**:

Fix id	Target	What is stale
`R-002a-architecture-pending-phases`	`docs/ARCHITECTURE.md`	The "On-Device Lava API" section's `**Pending (Phases C/D/E):**` note. Those phases **have** shipped — the note under-claims reality.
`R-002b-claude-md-project-proxy`	root `CLAUDE.md`	The `## Project` section still describes a companion "Ktor proxy server" and lists a `:proxy` Gradle module. `:proxy` is **not** in `settings.gradle.kts` and has no source tree (only a stale `build/` directory) — it was superseded by `lava-api-go` in SP-2. The real second Android artifact is `:api-app`.

Both fixes are **exact-string matched**, therefore idempotent: re-running after a successful pass is a no-op, and if the surrounding prose has since been rewritten by a human the fix simply does not apply and reports `NOOP`, rather than fuzzy-matching and mangling the document.

Each fix reports one verdict: `APPLIED`, `WOULD-APPLY` (dry run), `NOOP`, `MISSING-TARGET`, or `ERROR`. A unified diff of what changed (or would change) is printed and captured into the phase's evidence log.

PASS 2 (T044) — derived-export regeneration

Every `.md` this phase actually changed gets its `.html` + `.pdf` siblings regenerated through the **existing** `scripts/sync-markdown-exports.sh`, invoked as `scripts/sync-markdown-exports.sh --regenerate <file>`.

This phase does **not** reimplement the pandoc/weasyprint pipeline, the in-scope/excluded path rules, or the staleness definition. Reimplementing any of them would violate the same Local-Only CI/CD "no parallel implementation" principle the pipeline already relies on for `firebase-distribute.sh` (research R-003) and the systemd scripts (R-012).

Usage

```
scripts/pipeline/phase-06-docs.sh <run_id> [repo-path] [options]
```

Option	Meaning
`--dry-run`	print the exact unified diff each PASS-1 fix *would* apply; change nothing on disk (no doc edits, no exports, no Evidence Record, no `report.json` append)
`--skip-exports`	run PASS 1 only. For hosts without pandoc/weasyprint. Recorded honestly in the Evidence Record as *skipped*, not as passed
`--regenerate-all`	in PASS 2, additionally run the whole-repo `sync-markdown-exports.sh --regenerate-all` sweep instead of only this phase's changed files. Slow; opt-in

`<run_id>` must already have a `report.json` (created by `lib/run-report.sh`'s `init_run_report`). This script appends one `docs_refresh` phase entry to that report; it never creates a new run.

Examples:

```
# Review the stale-doc fixes before letting the pipeline apply
        them
scripts/pipeline/phase-06-docs.sh 2026-08-21T19-36-52Z --dry-run

# Apply the fixes but skip exports (no pandoc on this host)
scripts/pipeline/phase-06-docs.sh 2026-08-21T19-36-52Z --skip-
        exports
```

Scoping of PASS 2's verdict (important)

`sync-markdown-exports.sh --check-only` reports the state of the **whole repository**, and this repository routinely carries export staleness caused by other, unrelated in-flight work. Failing this phase for those pre-existing problems would make it report a failure it did not cause and cannot fix without rewriting files it never touched.

So the verdict is scoped precisely: the phase **fails if and only if a file this phase changed** still appears in `--check-only`'s problem list afterwards. Every other problem `--check-only` reports is printed verbatim into the evidence log and explicitly named as pre-existing — reported, never silently swallowed (§6.J), and never counted against this phase.

Anti-bluff properties

1. **Exact-match, never fuzzy.** `APPLIED` means the specific stale text was genuinely present and is now rewritten; `NOOP` means it was genuinely absent. There is no in-between "close enough" state.
2. **Post-run verification of the exports.** A regeneration step that exits 0 is not trusted on its own — `--check-only` is re-run afterwards and the phase fails if a changed file's siblings are still `MISSING`/`STALE`. This was proven by a falsifiability rehearsal: a sabotaged exporter that exits 0 while writing nothing is caught and fails the phase.
3. **Honest skips.** `--skip-exports` and dry-run record their non-execution in the Evidence Record's `assertion_summary` rather than reporting a pass.
4. **No vacuous verification claim.** Both post-run checks iterate over the files PASS 1 changed. `--regenerate-all` is the one flag that runs the export pass with that list **empty** (the docs were already correct but the operator asked for the whole-repo sweep anyway), and both loops then execute zero times. The `assertion_summary` states the number of files it examined, and when that number is zero it says so — "*...each examined ZERO files and verify nothing about this run's exports*" — instead of the vacuously-true "*every changed .md was verified first-hand*". It also discloses that the whole-repo `--regenerate-all` sweep's own output is **not** verified by this phase, because confirming it would mean duplicating `sync-markdown-exports.sh`'s in-scope path rules, which this phase refuses to do. Covered by `tests/pipeline/test_phase_06_regenerate_all_claim.sh`.

Exit codes

Code	Meaning
`0`	every PASS-1 fix applied-or-already-correct **and** (unless `--skip-exports`) every changed file has fresh `.html`+`.pdf` siblings confirmed by re-running `--check-only`; Evidence Record written and anti-bluff-validated. Under `--dry-run`: the full would-be diff was printed and nothing written
`1`	a real failure — a fix could not be applied although its stale text *was* present, a target file is missing, an export regeneration failed, a changed file is still missing/stale afterwards, or the Evidence Record was rejected. Recorded as `FAIL` in `report.json`, never fabricated as success
`2`	usage/precondition error — missing `run_id`, absent `report.json`, unknown option

Maintenance

When this script is modified, update this document in the same commit (§11.4.18 / `CM-SCRIPT-DOCS-SYNC` convention). When one of the two R-002 staleness findings is fixed permanently by a human, this script's corresponding fix becomes a permanent `NOOP` — that is the intended end state, and the fix should only be removed once the surrounding prose is stable enough that a future regression is implausible.

Cross-references

- `scripts/pipeline/phase-06-docs.sh` — the script itself
- `scripts/sync-markdown-exports.sh` — the export generator/checker this phase reuses (`docs/scripts/sync-markdown-exports.sh.md`)
- `scripts/pipeline/phase-05a-changelog-entry.sh` — the *pre*-distribution changelog phase
- `specs/002-build-test-distribute-pipeline/research.md` — R-002 (the staleness findings), R-003/R-012 (the no-parallel-implementation precedent)
- `docs/ARCHITECTURE.md`, root `CLAUDE.md` — the two documents this phase corrects